Università di Genova
DIBRIS DIPARTIMENTO
DI INFORMATICA, BIOINGEGNERIA,
ROBOTICA E INGEGNERIA DEI SISTEMI

Decentralized Project Management System with Blockchain and IPFS

Anita Ghezel Ayagh Tehrani

Master's Thesis

Università di Genova, DIBRIS
Via Dodecaneso, 35 16146 Genova, Italy
https://www.dibris.unige.it/



Computer Science MSc
Software Security and Engineering Curriculum

Decentralized Project Management System with Blockchain and IPFS

Anita Ghezel Ayagh Tehrani

Advisor: Marina Ribaudo

Examiner: Gianna Reggio

October, 2026

Abstract

Software project management platforms in wide use today — Trello, Jira, Asana, GitHub Projects, and GitLab among them — share a common limitation: the history of changes to a project is stored and controlled entirely by a single centralized provider, with no mechanism for participants to independently verify that it has not been altered. This thesis proposes and implements a decentralized project management system that addresses this limitation by combining Hyperledger Fabric, a permissioned blockchain framework, with the InterPlanetary File System (IPFS) for off-chain file storage, following an architectural pattern validated in prior work on decentralized data management in the construction industry and adapting it to the software project management domain.

The system, named ChainBoard, models accounts, projects and tasks as chaincode-governed state on a Fabric ledger, exposing operations for project creation, role-based membership, task assignment, and status transitions, all of which are permanently recorded in an immutable transaction history. The chaincode itself enforces who may do what, according to the roles of owner, administrator and contributor, so that the permission rules do not depend on the backend or the user interface. Files attached to tasks are stored on IPFS and referenced on-chain via their content identifiers, anchoring their exact content to the ledger without storing binary data directly in blockchain state. A backend API and browser-based frontend expose this functionality through a familiar task-board interface, while the audit trail view makes the system's core guarantee — a tamper-evident, independently verifiable record of every change, including who made it and when — directly visible to the user. A verification procedure checks every record of a history against the ledger and produces a downloadable report with a digest chain that can be re-verified offline.

A working prototype was implemented and evaluated end to end, from account creation, project and task creation through file attachment and status transitions, and complemented by an automated test suite, a performance evaluation and a threat model. The results show that the proposed architecture is a practical foundation for decentralized project management in software development contexts, and the evaluation states its limits explicitly, most importantly that the backend, and not Fabric, identifies the person acting.

Table of Contents

Chapter 1 Introduction **7**

1.1 Motivation 7

1.2 Research Questions and Objectives 8

1.3 Content of the Thesis 8

Chapter 2 Background **10**

2.1 Project Management 10

2.2 Traditional Project Management Platforms 11

2.3 Blockchain Technology 12

2.3.1 Smart Contracts 13

2.4 Hyperledger Fabric 13

2.5 Tamper-Evident Logs and Hash Chains 14

2.6 The InterPlanetary File System 14

Chapter 3 Related Work **16**

3.1 Decentralized Data Management with Hyperledger Fabric and IPFS 16

3.2 Other Blockchain-Based Data Management Systems 17

3.3 Tamper-Evident Logging 17

3.4 Performance of Hyperledger Fabric 18

3.5 Positioning of This Thesis 18

Chapter 4 System Design and Implementation **20**

4.1 System Architecture 20

4.2 Chaincode Design 22

4.2.1 Data Model and Keys 22

4.2.2 Project Functions 22

4.2.3 Task Functions 23

4.2.4 Role-Based Permissions Enforced by the Chaincode 24

4.2.5 Determinism Constraints 25

4.2.6 Ordering of the History 25

4.3 Accounts and Authentication 26

4.4 IPFS Integration 27

4.5 Backend API 27

4.6 Audit Verification 28

4.7 Frontend Interface 29

Chapter 5 Results **32**

5.1 Requirements Coverage 32

5.2 Chaincode Deployment and Basic Operations 32

5.3 Role-Based Permissions 34

5.4 Immutable Audit Trail and its Verification 34

5.5 IPFS File Attachment 35

5.6 End-to-End Workflow via the Backend API and Frontend 35

5.7 Automated Tests 36

5.8 Performance Evaluation 37

5.9 Threat Model 38

5.10 Discussion 39

Chapter 6 Conclusion **41**

6.1 Answers to the Research Questions 42
6.2 Limitations 42
6.3 Future Work 43

Appendix A Reproducing the Prototype and its Evaluation **44**
A.1 Starting the Network and the Application 44
A.2 Tests and Benchmark 44
A.3 Summary of the REST API 45

Chapter 1

Introduction

1.1 Motivation

Project management platforms such as Trello, Jira, Asana, GitHub Projects, and GitLab are the default tools used by software teams today to plan, assign, and track work (Section 2.2). These platforms are mature and effective at their core task, but they share an architectural property that is easy to overlook: every project record, every status change, and every audit trail they produce is stored and controlled entirely by a single provider. If a team wants to verify *independently* that a task's history has not been altered, there is no mechanism to do so — they can only trust that the platform has not modified it.

This matters in any collaborative setting where trust between participants cannot be assumed, or where an independently verifiable record of who did what, and when, is itself a requirement rather than a convenience. Multi-organization software projects, outsourced development work, and academic or regulatory contexts where accountability must be demonstrable are all cases where a centrally-controlled audit log is a weaker guarantee than participants may need.

Blockchain technology, and permissioned blockchain frameworks such as Hyperledger Fabric in particular (Section 2.4), were designed to solve exactly this class of problem: providing a shared, tamper-evident record of state changes among a known set of participants, without requiring any single participant to be fully trusted. Adel et al. [Ade+23] demonstrate that this approach is practical for collaborative data management by applying it to the construction industry (Section 3.1), combining Hyperledger Fabric for on-chain metadata with the InterPlanetary File System (IPFS) for off-chain document storage. However, their system targets a construction-specific data model that does not map onto how software teams organize their work.

This thesis addresses that gap: it applies the validated Fabric-plus-IPFS architectural pattern to the specific domain of software project management, producing a system that offers the functional surface software teams already expect — projects, tasks, priorities, assignments, status boards, file attachments — while replacing the centrally-controlled audit trail of existing tools with an immutable, independently verifiable one.

1.2 Research Questions and Objectives

The thesis is organized around four research questions.

RQ1 Can the Fabric-plus-IPFS pattern validated for construction data (Section 3.1) be adapted to software project management, offering the functions that software teams expect from tools such as Trello or Jira?

RQ2 Can the rules of who may do what in a project be enforced by the chaincode itself, so that they do not depend on the correctness of the backend or of the user interface?

RQ3 How can the recorded history of a task or project be made not only tamper-evident on the ledger but also *verifiable by a person* who does not have access to the network?

RQ4 What does this guarantee cost in latency and throughput, and where do its limits lie?

Accordingly, the objectives were to design a data model and chaincode for accounts, projects and tasks (RQ1); to enforce a role-based permission matrix in the chaincode (RQ2); to design and implement an audit verification procedure with a portable report (RQ3); and to measure the prototype and state a threat model (RQ4). Chapter 6 returns to these questions and answers them.

1.3 Content of the Thesis

Chapter 2 introduces the concepts needed to understand the rest of the document: the structure and requirements of software project management (Section 2.1), the limitations of existing centralized platforms (Section 2.2), and the technical foundations — blockchain, smart contracts, Hyperledger Fabric, and IPFS — that the proposed system builds on.

Chapter 3 discusses Adel et al. [Ade+23] in depth, the closest prior work to this thesis, and uses it to precisely position this thesis' contribution relative to both the blockchain-based

data management literature and the traditional project management tools discussed in Chapter 2.

Chapter 4 describes the system itself: its layered architecture (Section 4.1), the chaincode that governs account, project and task state on the Fabric ledger and enforces role-based permissions (Section 4.2), the handling of accounts and login sessions (Section 4.3), the integration with IPFS for file attachments (Section 4.4), the backend API that exposes this functionality over HTTP (Section 4.5), the audit verification mechanism that lets anyone re-check a recorded history (Section 4.6), and the frontend interface used to interact with it (Section 4.7). This chapter also discusses concrete implementation issues encountered during development — a chaincode determinism bug caused by including timestamps generated independently by each peer, and the ordering of ledger history — as illustrations of constraints that are easy to overlook when building Fabric-based systems.

Chapter 5 evaluates the resulting prototype against the requirements set out in Section 2.1, demonstrating the end-to-end workflow — from account creation and project creation through task assignment, status transitions, and file attachment — examining the permission rules and the verifiable audit trail the system produces as its central contribution, and reporting the automated tests, a performance evaluation and a threat model that states what the design does and does not protect against.

Finally, Chapter 6 answers the research questions of Section 1.2, summarizes what was achieved, discusses the limitations of the current prototype, and outlines directions for future work. Appendix A lists the steps needed to run the prototype and repeat the measurements.

The contributions of this thesis can be summarized as follows: (i) the adaptation of the Fabric-plus-IPFS pattern to software project management; (ii) a chaincode that enforces role-based permissions for every state-changing operation and stores accounts as ledger transactions; (iii) an audit verification procedure, based on a ledger check of every transaction and a digest chain that can be recomputed offline; and (iv) an evaluation that includes a threat model and an explicit account of the limits of the approach.

Chapter 2

Background

This chapter introduces the foundational concepts necessary to understand the system designed and implemented in this thesis. We begin with an overview of project management and its core phases, then examine existing centralized platforms and their limitations. We then introduce blockchain technology, with a focus on Hyperledger Fabric, and the InterPlanetary File System (IPFS), which together form the technical foundation of the proposed decentralized project management system described in Chapter 4.

2.1 Project Management

Project management is the discipline of organizing and directing tasks, people, and resources toward a defined goal, within constraints of time, budget, and quality [Pro21]. In software engineering contexts, it encompasses the full lifecycle of a software product, from initial planning through delivery and archival.

The standard project lifecycle is typically divided into five phases [Pro21]:

1. **Initiation** — defining the project scope, objectives, and stakeholders.
2. **Planning** — decomposing the work into tasks, assigning responsibilities, estimating effort, and setting timelines and dependencies.
3. **Execution** — carrying out the planned work, coordinating team members, and managing resources.
4. **Monitoring and Controlling** — tracking progress against the plan, identifying deviations, and making adjustments.

5. **Closing** — delivering the final product, archiving project artifacts, and capturing lessons learned.

The core functional requirements of a project management system therefore include: task creation and assignment, status tracking, role-based access control, file attachment, team communication, milestone tracking, and audit history. These requirements directly inform the design of the system presented in Chapter 4.

2.2 Traditional Project Management Platforms

A number of widely adopted platforms exist for software project management. Each takes a different approach to organizing work, but all share a common architectural characteristic: they are centralized, meaning all data is stored and controlled by a single provider.

Trello [Atl24b] uses a board-and-card metaphor inspired by Kanban [And10]. Projects are represented as boards, phases as lists, and individual work items as cards. It is well suited to simple workflows but offers limited support for complex dependencies or audit trails.

Jira [Atl24a] by Atlassian is the dominant tool in professional software development. It supports backlogs, sprints, epics, and detailed reporting. Its flexibility makes it powerful but complex to configure.

Asana [Asa24] emphasizes cross-team coordination, offering task dependencies, timelines, and multi-project views. It targets teams that need to manage work across several concurrent projects.

GitHub Projects and **GitLab** [Git24a; Git24b] integrate project management directly with version control, linking issues and milestones to code commits. This makes them natural choices for software teams but limits their applicability outside of code-centric workflows.

Despite their differences, all of these platforms share a fundamental limitation: *they rely on a trusted central authority.* All project data, task history, and access decisions are managed by the platform provider. This creates three categories of risk:

- **Auditability** — there is no independently verifiable, tamper-evident record of who changed what and when. Audit logs exist, but they are controlled by the platform and could in principle be altered.
- **Data sovereignty** — organizations have no control over where their data is stored or how it is accessed by the provider.

- **Single point of failure** — platform outages or discontinuation directly affect all dependent teams.

These limitations motivate the decentralized approach explored in this thesis, as discussed further in Chapter 3.

2.3 Blockchain Technology

A blockchain is a distributed ledger: a data structure that records a sequence of transactions across a network of nodes, where each transaction is cryptographically linked to the previous one [Nak08]. This linkage makes the history of transactions tamper-evident — altering any past record would require recomputing all subsequent blocks and obtaining consensus from the majority of the network.

Two broad categories of blockchain exist, distinguished by who may participate:

- **Public (permissionless) blockchains**, such as Bitcoin [Nak08] and Ethereum [But14], allow anyone to join and submit transactions. They achieve consensus through computationally expensive mechanisms such as Proof of Work or Proof of Stake.
- **Private (permissioned) blockchains** restrict participation to a known set of authenticated nodes. Consensus is reached through more efficient protocols, making them suitable for enterprise applications where participants are identified and trusted to a degree.

For a multi-party software project management system, a permissioned blockchain is the appropriate choice: participants are known (team members, organizations), throughput requirements are modest, and there is no need for anonymous participation.

A blockchain is not always the right tool, and it is fair to ask when it is. Wüst and Gervais [WG18] propose a decision scheme whose questions include whether state must be stored, whether several writers are involved, whether they trust each other, and whether a trusted third party is available or acceptable. A project tracker used by several organizations that do not fully trust one another, and which needs a history that no single operator can rewrite, answers these questions in favor of a (permissioned) blockchain; a tracker used by a single team that trusts its own administrator does not, and a conventional database is simpler, faster and cheaper there. This thesis is concerned with the first kind of setting.

2.3.1 Smart Contracts

A smart contract is a program that runs on the blockchain and encodes the rules governing how the ledger state may be modified [Sza97]. When a participant submits a transaction, the smart contract validates it against predefined rules and, if valid, updates the ledger state. Because the contract code is deployed to and executed by the blockchain network, its logic is transparent, auditable, and cannot be unilaterally altered by any single party. In the system presented in this thesis, smart contracts implement all project and task management operations, as described in Chapter 4.

2.4 Hyperledger Fabric

Hyperledger Fabric [Hyp24; And+18] is an open-source permissioned blockchain framework hosted by the Linux Foundation. It is designed for enterprise use cases where participants are identified, privacy between subsets of participants may be required, and high transaction throughput is important.

The key architectural components of Fabric relevant to this thesis are:

- **Organizations and peers** — the network is formed by one or more organizations, each operating one or more peer nodes that maintain a copy of the ledger.
- **Orderer** — a dedicated node (or cluster) responsible for ordering transactions into blocks and distributing them to peers.
- **Channels** — a channel is a private subnet of communication between a subset of organizations, each with its own independent ledger. This enables data isolation between groups of participants.
- **Chaincode** — Fabric's term for smart contracts. Chaincode is deployed to the network and executed by peers when transactions are submitted. It is written in general-purpose languages (Go, JavaScript, or Java).
- **World state and ledger history** — Fabric maintains two representations of data: the *world state*, a key-value store holding the current value of each record; and the *blockchain history*, the immutable sequence of all past transactions that led to the current state. The history cannot be modified, providing a permanent audit log.
- **Membership Service Provider (MSP)** — manages identity and access control, ensuring that only authorized participants can submit transactions or query the ledger. Identities are X.509 certificates issued per organization or, if desired, per user; a chaincode can read the identity of the caller of a transaction.

- **Endorsement and concurrency** — a transaction is simulated by the endorsing peers, which must all produce the same result, and is then ordered and validated. Validation uses optimistic concurrency control: if two transactions in the same block read and write the same key, only the first is committed and the other is rejected with a read conflict. The data model of a Fabric application should therefore avoid keys that many users update at once.

Fabric was selected for this thesis over alternatives such as Ethereum because its permissioned model, support for private channels, and chaincode flexibility make it well suited to a collaborative project management context where participants are known and data privacy between organizations may be required.

2.5 Tamper-Evident Logs and Hash Chains

Making a history tamper-*evident* (rather than tamper-proof) is an old problem in computer security. Schneier and Kelsey [SK99] describe audit logs in which every entry is cryptographically bound to the ones before it, so that an attacker who gains control of the machine later cannot alter earlier entries without detection. The basic building block is a *hash chain*: each entry stores, or is hashed together with, the digest of the previous entry, so that changing, removing, inserting or reordering any entry changes every digest after it. Crosby and Wallach [CW09] improve on the cost of checking such logs with tree-based structures, and Certificate Transparency [LLK13] applies append-only, publicly auditable logs, built on Merkle trees, to the issuing of TLS certificates. In all of these the useful property is the same: a short value (the last digest or the root of a tree) commits to the entire history, so that anyone who remembers or has published that value can later detect a change.

A blockchain is a replicated, consensus-protected log of this kind: each block contains the hash of the previous one. The audit verification of Section 4.6 builds a second, much smaller hash chain, over the history of a single task or project, using SHA-256 [Nat15]. Its purpose is not to replace the blockchain's own protection but to give a person a portable fingerprint that can be checked offline.

2.6 The InterPlanetary File System

The InterPlanetary File System (IPFS) [Ben14] is a peer-to-peer distributed file system designed for content-addressed storage. Unlike traditional systems that locate files by their

URL (i.e., *where* a file is), IPFS identifies files by their *content*: each file is hashed, and the resulting hash — called a Content Identifier (CID) — serves as its unique address.

This design has two important properties for the purposes of this thesis:

- **Integrity** — because the CID is derived from the file content, any modification to the file produces a different CID. A stored CID therefore constitutes a cryptographic proof of a file's content at a given point in time.
- **Decentralization** — files are distributed across the IPFS network and can be retrieved from any node that holds a copy, eliminating dependence on a central file server.

In the system proposed in this thesis, project documents are uploaded to a local IPFS node. The resulting CID is then recorded on the Fabric ledger as part of a transaction, anchoring the document to the immutable blockchain history. This separation of concerns — metadata and audit trail on-chain, file content off-chain — is a common pattern in blockchain-based systems [Ade+23] and avoids storing large binary data directly on the ledger, which would be inefficient. The integration is described in detail in Chapter 4.

Chapter 3

Related Work

The centralized project management platforms discussed in Section 2.2 are mature and widely adopted, which raises an obvious question: hasn't the problem of managing software projects already been solved? This chapter argues that it has not, at least not with respect to the specific guarantee this thesis targets — an auditable, tamper-evident history of project and task changes that does not depend on trusting a single platform provider. We review the closest prior work addressing this gap with blockchain technology, and use it to position the contribution of this thesis.

3.1 Decentralized Data Management with Hyperledger Fabric and IPFS

The most closely related work is Adel et al. [Ade+23], who propose a decentralized data management system for the construction industry built on Hyperledger Fabric and IPFS. Their setting is structurally similar to the one addressed in this thesis: construction projects involve multiple independent parties (contractors, architects, engineers) who need to share task metadata and documents while maintaining a verifiable, tamper-evident record of changes, without relying on a centralized authority.

Adel et al. [Ade+23] adopt the same architectural separation used in this thesis: Hyperledger Fabric stores structured metadata and task and progress records on-chain, giving an immutable history of updates, while large binary assets — drawings, images, and documents — are stored on IPFS and referenced from the ledger via their content identifiers (CIDs). This avoids the inefficiency of storing large files directly in blockchain state, while still cryptographically binding any file reference to its exact content. They also use Fabric's channel and private data collection features to restrict certain information to subsets of

participants, and implement chaincode functions that handle project updates, task assignments, and asset tracking as the sole mechanism through which ledger state may change.

This work is directly relevant to the present thesis for three reasons. First, it demonstrates that the Fabric-plus-IPFS architecture is a viable, previously validated pattern for decentralized project data management rather than a purely theoretical combination of technologies. Second, it establishes the on-chain metadata / off-chain file convention that this thesis follows for attaching documents to tasks (Chapter 4). Third, and most importantly for motivating this thesis, Adel et al. target the construction domain, where the task model — physical progress milestones, drawings, regulatory documents — differs substantially from the software task model this thesis addresses (issues, priorities, status transitions across a todo/in-progress/done workflow, code-adjacent artifacts). Our work is similar to that of Adel et al. in its underlying architecture, but we address a different task model and target software teams already familiar with tools such as Jira or GitHub Projects, which imposes different requirements on the chaincode's data model and the user-facing workflow (Chapter 4).

3.2 Other Blockchain-Based Data Management Systems

Blockchain-based data management has also been explored outside construction, notably in supply chain traceability and healthcare records. These domains share the same underlying requirement addressed by Adel et al. [Ade+23] and by this thesis: multiple mutually distrusting parties need a shared, verifiable history of state changes. Such systems typically favor permissioned blockchains over permissionless ones, for the reasons discussed in Section 2.3 — participants are known in advance, throughput requirements are moderate, and selective privacy between organizations is often required. While their data models differ substantially from project management (tracking a physical product's custody chain, for instance, rather than a software task's lifecycle), the recurring pattern of anchoring off-chain content via content-addressed identifiers on an immutable ledger reinforces the architectural choice discussed in Section 3.1 and adopted in Chapter 4, rather than introducing a fundamentally different approach.

3.3 Tamper-Evident Logging

Independently of blockchains, tamper-evident logging is a well-studied topic (Section 2.5). Schneier and Kelsey [SK99] and Crosby and Wallach [CW09] study logs whose integrity can be checked after an attack, and Certificate Transparency [LLK13] shows the approach

at internet scale. These systems protect a *log*, usually a single append-only sequence kept by one operator and checked by auditors. ChainBoard differs in two respects: the history is held by a replicated, consensus-based ledger operated by several organizations rather than by one operator, and a history exists *per task and per project*, tied to the business object a user is looking at. The digest chain of Section 4.6 borrows the hash-chain idea from this literature, at the scale of one object, in order to make a downloaded copy of a history checkable offline.

3.4 Performance of Hyperledger Fabric

The performance of Fabric has been studied in its own right. Thakkar, Nathan, and Viswanathan [TNV18] benchmark the platform and show that throughput and latency depend strongly on factors such as the block size and timeout of the ordering service, the endorsement policy, the number of concurrent clients, and conflicts between transactions that touch the same keys. These observations shape both the data model of this thesis, which gives every account, project and task its own key so that unrelated writes do not conflict, and the way its measurements are interpreted (Section 5.8): the figures reported here describe one machine and a synthetic load, and are meant to show the order of magnitude and the behavior under contention, not to establish the platform's limits.

3.5 Positioning of This Thesis

Traditional platforms such as Trello, Jira, Asana, GitHub Projects, and GitLab (Section 2.2) offer mature, feature-rich project management functionality but no cryptographic guarantee of history integrity: where audit logs exist, they are controlled entirely by the platform provider, and could in principle be altered without independent verification. Adel et al. [Ade+23] show that a Fabric-plus-IPFS architecture can close this gap, but do so in a domain-specific context whose data model does not transfer directly to software project management.

This thesis addresses the resulting gap by adapting the validated Fabric-plus-IPFS pattern to the software project management domain: projects, tasks, role-based membership, and status transitions are modeled as chaincode-governed state, while the system preserves the functional surface — task boards, assignments, priorities, file attachments — that users of tools such as Trello or Jira already expect. Beyond the adaptation of the data model, the thesis adds three elements that the discussion above does not cover: permissions that are enforced by the chaincode itself according to member roles, user accounts that are recorded as ledger transactions, and an audit verification procedure whose result can be re-checked

offline, in the spirit of tamper-evident logging (Section 3.3). It also states explicitly where the guarantees end (Section 5.9). The system design and implementation are described in Chapter 4.

Chapter 4

System Design and Implementation

This chapter describes the design and implementation of ChainBoard, the decentralized project management system proposed in this thesis. We first present the overall architecture (Section 4.1), then describe each layer in turn: the chaincode that governs on-chain state, including the role-based permissions it enforces (Section 4.2); the way user accounts and sessions are handled (Section 4.3); the integration with IPFS for off-chain file storage (Section 4.4); the backend API that bridges client applications and the Fabric network (Section 4.5); the audit verification mechanism that lets anyone re-check a recorded history (Section 4.6); and the frontend interface used to interact with the system (Section 4.7). The architectural choices follow directly from the requirements identified in Section 2.1 and the technologies introduced in Chapter 2, and mirror the on-chain/off-chain separation validated by Adel et al. [Ade+23] (Section 3.1), adapted to the software project management data model.

4.1 System Architecture

The system follows a layered architecture in which each layer has a single, well-defined responsibility:

1. **Frontend** — a browser-based client (Section 4.7) through which users sign in, create and inspect projects and tasks, and verify their audit trails.
2. **Backend API** — an HTTP service (Section 4.5) that authenticates users, translates REST requests into Fabric transactions and IPFS operations, and builds audit reports.

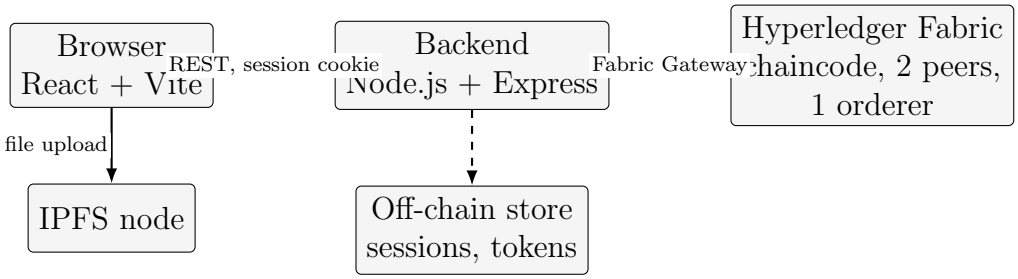


Figure 4.1: Architecture of ChainBoard. The browser talks only to the backend, apart from uploading file content to IPFS; the backend is the only component that holds the organization's Fabric identity. The dashed link is the short-lived operational data that stays off the ledger.

3. **Hyperledger Fabric network** — a permissioned blockchain network of two organizations, each running one peer, and one ordering service, connected by a single channel. This network hosts the chaincode (Section 4.2) that defines and enforces all business logic and permission rules.

4. **IPFS node** — a local content-addressed storage node (Section 4.4) used to store file attachments outside the ledger.

Figure 4.1 shows the layers and how they communicate.

A small *off-chain store* inside the backend holds only short-lived operational data: login sessions, e-mail confirmation tokens and rate-limit counters (Section 4.3). Everything that has business meaning, including the user accounts themselves, is on the ledger.

The request flow for a typical write operation — for example, creating a task — proceeds as follows: the frontend sends an HTTP request to the backend; the backend identifies the signed-in user from the session cookie, constructs and submits a Fabric transaction using the Fabric Gateway client SDK, passing that user as the *acting person*; the transaction is endorsed by the peers of both organizations according to the channel's endorsement policy, ordered by the orderer, and committed to the ledger on both peers; the backend returns the resulting state to the frontend as a JSON response. Read operations follow the same path but use Fabric's `evaluateTransaction` call, which queries the local peer's world state directly without going through the ordering service, since no ledger update is required.

File attachment follows a variant of this flow: the file is first uploaded directly from the frontend to the local IPFS node, which returns a CID; this CID, together with the file name, is then submitted to the chaincode as part of a normal transaction, so that only the CID — not the file content — is recorded on the ledger (Section 4.4).

4.2 Chaincode Design

The chaincode is implemented in JavaScript using the Fabric Shim API and deployed to a channel shared by both organizations in the network. It defines three primary record types, distinguished by a `docType` field: *users*, *projects* and *tasks*. Each record is stored under its own key in Fabric's world state, and every write to a key is simultaneously appended to that key's immutable transaction history, which Fabric retains independently of the current state (Section 2.4).

4.2.1 Data Model and Keys

A *project* record holds an identifier, name, description, owner, a list of members each with a role (`owner`, `admin` or `contributor`), and a status (`active` or `archived`). A *task* record holds an identifier, a reference to its parent project, a title, description, priority (`low`, `medium`, `high`), an optional due date, a status (`todo`, `in-progress`, `done`), an optional assignee, an archived flag, a list of file attachments and a list of comments. Every project and task record also carries an `updatedBy` field naming the person who made the latest change, so that each version in the history states *who* produced it.

The default world-state database of Fabric, LevelDB, offers key lookups and key-range scans but no rich queries. The data model is therefore designed around keys: one key per task (`task:<id>`), per project (`project:<id>`) and per user (`user:<email>`), plus *index keys* written in the same transaction as the record they describe. Index keys are used for uniqueness checks (Section 4.3). The tasks of a project are retrieved by a range scan over the `task:` prefix, filtering on the project identifier, which is what allows a board to be rebuilt from the ledger alone rather than from a list remembered by one browser.

4.2.2 Project Functions

The chaincode exposes the following project functions. As explained in Section 4.2.4, every state-changing function takes the acting person as its last argument.

- `createProject(projectId, name, description, ownerId)` — creates a new project with the owner as its sole initial member, rejecting the call if the identifier is already in use.
- `getProject(projectId)` — returns the current state of a project.
- `addProjectMember(projectId, memberId, role, actorId)` — adds a member with a given role to an active project, rejecting duplicates.

- `archiveProject(projectId, actorId)` — marks a project as archived; an archived project accepts no new members and no new tasks, and its history remains on the ledger.
- `getProjectTasks(projectId)` — returns every task of a project through the range scan described above.
- `getProjectHistory(projectId)` — returns the full sequence of past states of a project (Section 4.2.6).

4.2.3 Task Functions

- `createTask(taskId, projectId, title, description, priority, dueDate, actorId)` — creates a task within an existing, active project.
- `getTask(taskId)` — returns the current state of a task.
- `assignTask(taskId, assigneeId, actorId)` — assigns a task to a project member; the chaincode verifies that the assignee is listed among the parent project's members, and rejects reassignment of a completed task.
- `updateTaskStatus(taskId, newStatus, actorId)` — transitions a task between statuses. Transitions are restricted to a fixed set of valid moves: `todo` → `in-progress`; `in-progress` → `todo` or `done`; and a finished task can be reopened to `in-progress` or `todo`. A task therefore cannot jump directly from `todo` to `done`, and every reopening remains visible in its history.
- `updateTaskMeta(taskId, field, value, actorId)` — edits the title, description, priority or due date of a task that is not yet completed.
- `addComment(taskId, authorId, text)` — appends a comment; the author is the acting person.
- `archiveTask(taskId, actorId)` — marks a task as archived. A ledger cannot delete history, so "deleting" a task means removing it from the active board while its full history stays on the chain.
- `attachFile(taskId, fileName, ipfsCid, actorId)` — records a reference to an off-chain file (Section 4.4).
- `getTaskHistory(taskId)` — returns the full immutable history of a task, analogous to `getProjectHistory`.

Action	Owner	Admin	Contributor
Add a member	any role	contributors only	no
Archive the project	yes	no	no
Create a task	yes	yes	yes
Assign a task	anyone	anyone	only to themselves, if the task is free
Change status, edit, attach a file	any task	any task	own or unassigned tasks
Archive a task	yes	yes	no
Comment	yes	yes	yes

Table 4.1: Permissions enforced by the chaincode. A person who is not a member of the project can do none of these actions. An archived project accepts no new members and no new tasks.

4.2.4 Role-Based Permissions Enforced by the Chaincode

Who may do what inside a project is decided by the chaincode, not by the backend or the user interface. Every state-changing function receives the identifier of the acting person as its last argument and checks it against the member roles stored in the project record. The rules are summarized in Table 4.1.

?? 4.1 shows the core of the implementation: a small set of pure helper functions that derive a person's role from the project record and decide whether a task may be changed, followed by the check performed by `addProjectMember`. Every other state-changing function starts with the same kind of check.

Listing 4.1: Role helpers and the check in `addProjectMember` (simplified from the chaincode).

```
function roleOf(project, id) {
    const found = (project.members || []).find(m => m.id === id);
    return found ? found.role : null;
}
function isManager(role) { return role === 'owner' || role === 'admin'; }
function canWorkOnTask(role, task, actorId) {
    if (!role) return false;
    return isManager(role) || !task.assigneeId || task.assigneeId ===
       actorId;
}

// inside addProjectMember(projectId, memberId, role, actorId)
const actorRole = roleOf(project, actorId);
if (!isManager(actorRole))
```

```
    return shim.error('Permission denied: only project owners and admins
        can add members');
if (actorRole === 'admin' && role !== 'contributor')
    return shim.error('Permission denied: admins can only add
        contributors');
```

A refusal is returned as an error whose message starts with "Permission denied", which the backend maps to HTTP status 403, and the frontend uses the same rules to disable buttons and to explain why an action is unavailable. The user interface is therefore only a convenience: removing it would not weaken the guarantee, because the ledger refuses the transaction anyway.

An important design limit must be stated explicitly. Fabric's gateway authenticates the *client organization*, not each end user: all transactions submitted by the backend carry the same organization identity. The chaincode can therefore enforce the *rules* (who may do what), but it has to trust the backend to say *who* is acting. The acting person is always taken from the verified login session and never from the request body, which is covered by automated tests, but a compromised backend could claim any identity. This trade-off is discussed together with the other threats in Section 5.9, and issuing one Fabric certificate per person is the natural remedy (Section 6.3).

4.2.5 Determinism Constraints

An important implementation constraint of chaincode is that its execution must be deterministic: every endorsing peer executes the same transaction independently and must produce byte-identical results for the transaction to be validated (Section 2.4). An early version of the chaincode included a server-generated timestamp (`new Date().toISOString()`) in each record. Because peers execute the same chaincode invocation at slightly different wall-clock times, this produced different output on different peers, causing endorsement mismatches and transaction failures. The timestamp fields were removed from the chaincode logic. Where a time is needed, it comes from Fabric itself: the account creation time is read from the transaction timestamp, which is identical on every peer, and the time of each historical record is the commit time stored in the ledger's history. For the same reason the chaincode never uses randomness and never computes hashes; password hashing is done in the backend (Section 4.3), and the chaincode only stores the result.

4.2.6 Ordering of the History

The history functions are built on Fabric's `getHistoryForKey` API, which returns the versions of a key from the newest to the oldest. Record numbers, the description of what

changed between two versions and the digest chain of Section 4.6 all need the opposite, chronological order. The history functions therefore sort the entries by their commit time (seconds and nanoseconds) before returning them, oldest first, adding to each entry its transaction identifier and its timestamp; the user interface then displays them newest first. Entries are identified by the camel-case field names (`txId`) used by the Node.js chaincode library; this detail matters because the in-memory ledger used for testing uses different names, which hid a defect in an earlier version until the system was run against a real network.

4.3 Accounts and Authentication

A distinctive choice of ChainBoard is that user accounts and every change to them are themselves blockchain transactions. An account record holds the name, e-mail address, username, phone number, the *hash* of the password, the linked Google identifier if any, and the list of projects the person belongs to. Account creation, password changes, linking or unlinking a Google account, and e-mail confirmation are all recorded as transactions, so the history of an account (with secrets removed) is a tamper-proof security log that the person can read in the application.

Uniqueness of username, phone number and Google identifier is guaranteed by the index keys introduced above: they are written in the same transaction as the account, and Fabric's multi-version concurrency control rejects the second of two simultaneous registrations that try to claim the same key.

4.3.0.0.1 What stays off the ledger. Passwords are hashed with the scrypt key-derivation function [Per09; PJ16] by the backend, which stores only the salted hash on the ledger; the chaincode could not compute it, since a salted hash is not deterministic (Section 4.2.5). Login sessions are an HttpOnly, SameSite cookie holding a random token; only the SHA-256 digest of the token is stored, in the backend, together with e-mail confirmation tokens, a log of sent e-mails and rate-limit counters. These items are short-lived and high-churn and carry no business value, so recording them on a blockchain would add cost without adding assurance.

4.3.0.0.2 Sign-in methods and hardening. People sign in with their e-mail address or username, or with Google: a Google ID token is verified on the server, and a Google sign-in never takes over an existing password account. Confirming the e-mail address is optional and does not block use of the application. State-changing requests are checked against the `Origin` and `Host` headers to resist cross-site request forgery, cross-origin resource sharing is

not enabled, responses carry security headers and are not cacheable, and sign-in, sign-up, username look-ups, Google endpoints and outgoing e-mail are rate limited. Failed sign-ins look identical whether the account exists or not, and a dummy hash is verified for unknown accounts so that response time does not reveal which accounts exist.

4.4 IPFS Integration

Following the pattern established by Adel et al. [Ade+23] (Section 3.1), file content is never stored on the Fabric ledger. Instead, files are uploaded directly from the client to a local IPFS node via its HTTP API, which returns a content identifier (CID) computed as a cryptographic hash of the file's content (Section 2.6). Only this CID, together with the original file name, is submitted to the chaincode's `attachFile` function and recorded as part of the task's on-chain state.

This design has two consequences that are relevant to the guarantees the system provides. First, because the CID is a function of file content, any subsequent modification of the file would produce a different CID; the CID recorded on the immutable ledger therefore constitutes a permanent, verifiable commitment to the exact content of the file at the time of attachment. Second, storing only a small fixed-size identifier on-chain, rather than the file itself, avoids the scalability and cost problems that would result from storing arbitrary binary data directly in blockchain state.

Retrieval works in the reverse direction: given a task record fetched from the chain, its attachments' CIDs can be used to fetch the corresponding files from any IPFS node holding a copy, including the local node's HTTP gateway, without any additional interaction with the Fabric network. Content addressing proves what a file contained, but it does not oblige anyone to keep storing it; pinning the files is an operational matter that this prototype does not address.

4.5 Backend API

The backend is an HTTP service built with Node.js and Express that exposes the chaincode's functionality as a conventional REST API, so that client applications do not need to interact with the Fabric Gateway SDK directly. Each request that reaches the ledger establishes a gRPC connection to a peer using the Fabric Gateway client, authenticated with the organization's X.509 certificate and private key, submits or evaluates the corresponding chaincode transaction, and returns the result as JSON. All access to the ledger goes through one small adapter module, which keeps the rest of the code independent of Fabric and makes it possible to run the whole backend against an in-memory ledger in

tests.

The API exposes endpoints mirroring the chaincode's function surface, for example `POST /api/projects` for `createProject`, `GET /api/projects/{id}/tasks` for `getProjectTasks`, `PUT /api/tasks/{id}/status` for `updateTaskStatus`, and `GET /api/tasks/{id}/history` for `getTaskHistory`, together with the authentication endpoints and the audit endpoints of Section 4.6. Write operations map to Fabric's `submitTransaction` call, and read-only operations map to `evaluateTransaction`, preserving the distinction between operations that require network-wide ordering and consensus and those that can be answered from local peer state alone (Section 2.4).

The backend keeps no business rules of its own: validation of status transitions, membership, assignments and permissions is enforced by the chaincode (Section 4.2.4), so these guarantees do not depend on the correctness of the backend. What the backend *does* contribute is identity: it authenticates the person, derives the acting person from the verified session, and reports it to the chaincode. For every state-changing route the owner, the comment author and the acting person come from the session and any such value in the request body is ignored. Chaincode refusals are translated into appropriate HTTP status codes: 403 for a permission refusal, 404 for an unknown record, 409 when Fabric's optimistic concurrency control reports that the ledger was busy, and 400 for invalid input.

4.6 Audit Verification

The ledger keeps the history of every task and project, but a user who only sees a list on a web page still has to trust the application that displays it. ChainBoard therefore adds a verification step that anyone can repeat, available as a "Verify audit trail" action on every task and project.

4.6.0.0.1 Ledger check. For every record of the history, the backend asks the peer's query system chaincode (`qscc`, function `GetTransactionByID`) whether the transaction that produced it is present in a committed block. A record that the peer does not know is reported as missing; if the peer cannot be asked, the record is reported as *not checked* rather than as missing, so that a connection problem is never mistaken for tampering.

4.6.0.0.2 Digest chain. The backend also computes a fingerprint of the whole history. Let v_i be the value of the i-th record ($i = 1, \ldots, n$, oldest first), t_i its transaction identifier and $\mathrm{canon}(\cdot)$ a canonical JSON serialization in which object keys are sorted at every level. Then

$$d_0 = 0^{64}, \qquad d_i = H(d_{i-1} \,\|\, t_i \,\|\, \mathrm{canon}(v_i)) \quad (i = 1, \ldots, n), \tag{4.1}$$

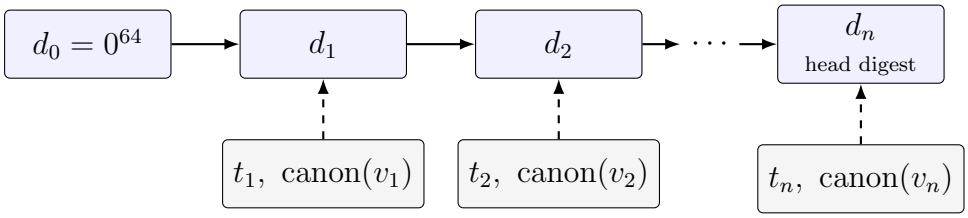


Figure 4.2: The digest chain. Each digest d_i is the SHA-256 hash of the previous digest, the transaction identifier t_i and the canonical serialization of the record v_i. Altering, removing, inserting or reordering any record changes the head digest.

where H is SHA-256, $\|$ denotes concatenation with a separator, and d_0 is a fixed starting value of 64 zero characters. The last digest d_n, the *head digest*, changes if any record is altered, removed, inserted or reordered, because every digest depends on the previous one. The head digest is a short value that can be written down and compared later.

Figure 4.2 illustrates the construction.

4.6.0.0.3 Offline verification. The complete report (the records, their digests, the ledger-check result and the head digest) can be downloaded as a JSON file. A small stand-alone script recomputes every digest from the file and reports which record, if any, no longer matches, without needing the application or the network. The digest chain is not a replacement for Fabric's own hash chain of blocks; it is a portable fingerprint that makes a copy of the history checkable by someone who has no access to the network.

4.7 Frontend Interface

The frontend is a single-page application written with React and Vite. After signing in, the person reaches a dashboard listing the projects they belong to, with their role in each, a search box and a status filter. A project page shows its members and roles and the project's history; the board shows the tasks of a project read from the ledger in three columns (to do, in progress, done), which can be filtered by text, assignee, priority and due date, and where moving a card between columns performs a status transition; and the task page shows a task's details, files, comments and activity. Separate pages show the person's profile and their settings (theme, text size, reduced motion and automatic refresh).

Figures 4.3 to 4.6 show the main screens.

Three design decisions are relevant to the thesis. First, the interface mirrors the chaincode's

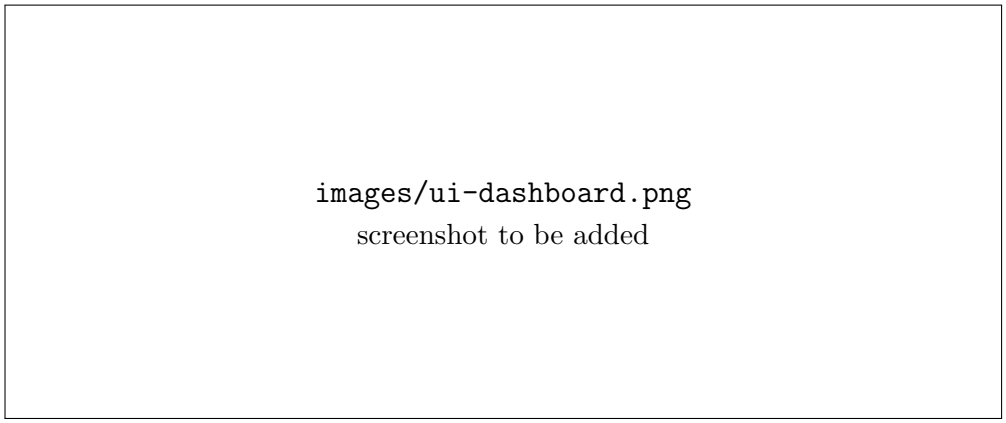


Figure 4.3: The dashboard: the projects a person belongs to, with their role, a search box and a status filter.

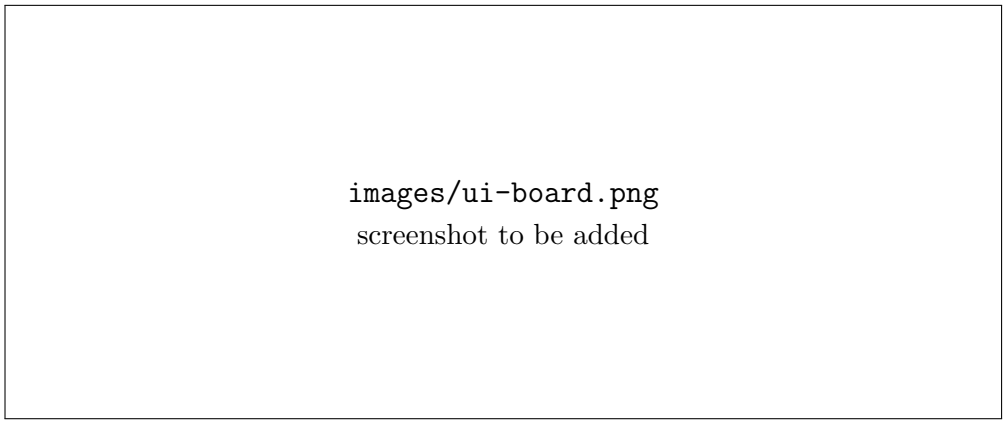


Figure 4.4: The task board read from the ledger, with filters by text, assignee, priority and due date.

permission rules, disabling an action and giving the reason when the person's role does not allow it, while the chaincode remains the authority (Section 4.2.4). Second, the history views make the central guarantee visible: rather than showing only the current state, each past record is rendered with what changed, who changed it and when, together with the "Verify audit trail" action of Section 4.6, so that a non-technical user can see that the record of how a task reached its current state cannot be altered after the fact, in contrast to the centralized platforms discussed in Section 2.2. Third, the screens keep themselves up to date by polling the backend at a modest rate, pausing while a change is being sent or edited, and while a request that changes data is in flight, every button and form is locked so that nothing can be submitted twice. The decision logic of each screen is kept in plain functions separate from the components, which is what makes it possible to unit test it without a browser.

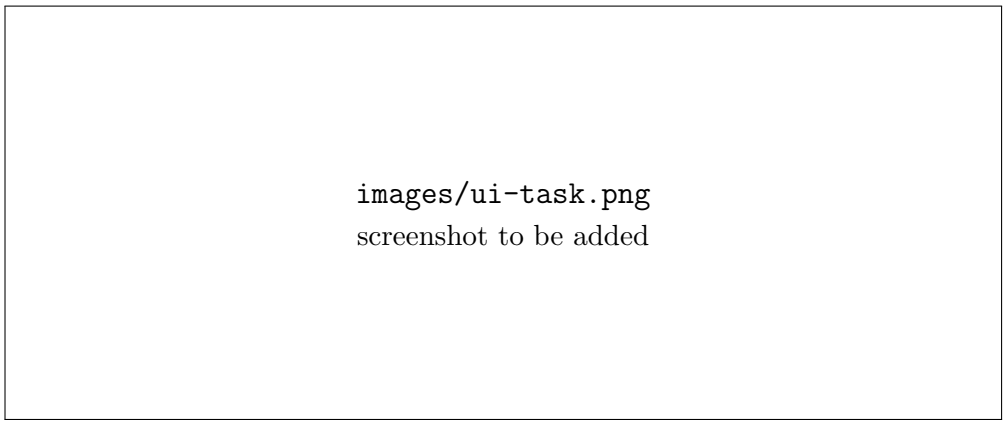


Figure 4.5: The task page: details, files, comments and the activity list with who changed what and when.

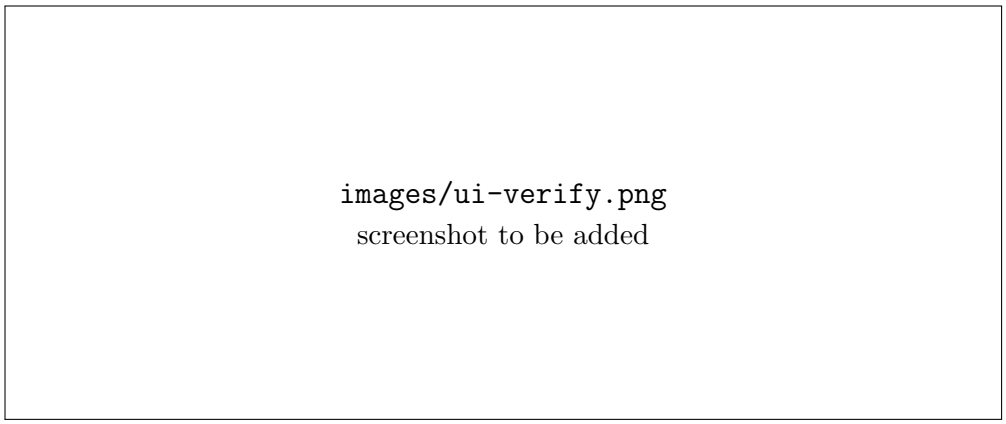


Figure 4.6: The result of the “Verify audit trail” action: every record is found in a committed block, and the head digest is shown.

Chapter 5

Results

This chapter evaluates the prototype described in Chapter 4 against the functional requirements identified in Section 2.1: project creation and archiving, role-based membership and permissions, task creation and assignment, status transitions, file attachment, and an immutable and verifiable audit history. Each requirement is demonstrated through the deployed system running on a local two-organization Hyperledger Fabric network with a connected IPFS node, exercised end to end through the REST API and the browser frontend described in Sections 4.5 and 4.7. Section 5.1 maps each requirement to the evidence. The chapter then reports the automated test suite (Section 5.7), the performance evaluation (Section 5.8) and an analysis of what the design does and does not protect against (Section 5.9).

5.1 Requirements Coverage

Table 5.1 restates the requirements of Section 2.1 in the form used for the evaluation (R1–R9), says where each one is implemented, and points to the section that reports the evidence. It also makes the link with the research questions of Section 1.2 explicit: R1, R3–R6 and R9 concern RQ1, R2 concerns RQ2, and R7–R8 concern RQ3.

5.2 Chaincode Deployment and Basic Operations

The chaincode described in Section 4.2 was packaged, installed on both peers, approved by both organizations, and committed to the channel using Fabric's standard lifecycle process (install, approve, commit). Because the signatures of the state-changing functions changed

ID	Requirement	Implemented in	Evidence
R1	Create and archive projects	Section 4.2	Section 5.2
R2	Roles, and rules of who may do what, enforced by the ledger	Section 4.2.4	Section 5.3
R3	Create tasks and assign them to members	Section 4.2	Sections 5.2 and 5.3
R4	Status workflow with valid transitions only	Section 4.2	Sections 5.2 and 5.7
R5	File attachments anchored by content identifier	Section 4.4	Section 5.5
R6	Comments by project members	Section 4.2	Section 5.6
R7	Immutable history showing what changed, who changed it and when	Section 4.2.6	Section 5.4
R8	History that a person can verify independently of the application	Section 4.6	Section 5.4
R9	User accounts and sessions, with account events recorded on the ledger	Section 4.3	Sections 5.6 and 5.7

Table 5.1: Requirements, where they are implemented and where the evidence is reported.

when permissions were added (Section 4.2.4), a new version has to be deployed together with the backend; on the development network this is done most simply by recreating the network. Following deployment, the core project and task functions were exercised directly through the Fabric peer CLI to confirm correct behavior at the chaincode layer, independently of the backend and frontend.

A project was created via `createProject`, and `getProject` confirmed that the resulting record correctly stored the owner as the sole initial member and set the project status to `active`. Calling `addProjectMember` on the same project appended a second member to the `members` array, and a subsequent `getProject` call confirmed the update was reflected in the current world state. A task was then created within the project via `createTask`, initialized with status `todo` and no assignee; `assignTask` correctly attached one of the project's members to the task, and `updateTaskStatus` moved the task from `todo` to `in-progress`, both changes being immediately visible via `getTask`.

An early version of the chaincode failed during endorsement with a mismatch error between

the two organizations' peers, traced to the non-deterministic timestamp issue discussed in Section 4.2.5. After removing the timestamp fields from the write logic, all subsequent transactions endorsed and committed successfully on both peers, confirming the fix.

5.3 Role-Based Permissions

The permission rules of Table 4.1 were verified with a seeded demonstration project and with automated tests, using five accounts: an owner, an administrator, two contributors and an outsider. The tests exercise the backend API, which in turn runs the real chaincode, and cover the following behavior.

- A person outside the project cannot create tasks, comment, change a task or add members; every attempt is answered with HTTP status 403 and nothing is written to the ledger.
- A contributor cannot add members; an administrator can add contributors but not administrators or owners; only the owner can grant any role.
- A contributor can take an unassigned task but cannot assign it to somebody else, and cannot take over a task already assigned to another person; the assignee, an administrator and the owner can change it.
- Only owners and administrators can archive a task, and only the owner can archive the project, after which no member or task can be added.
- A call without an acting person is refused by the chaincode itself, whatever the backend does, and a request body that names a different acting person is ignored.

Since a refused transaction is not committed, a refusal leaves no trace in the history; this is by design, because the history records changes of state and not attempts.

5.4 Immutable Audit Trail and its Verification

The central contribution of this thesis is the availability of a complete, tamper-evident history for every project and task, rather than only their current state. This was verified by calling `getTaskHistory` on a task that had been created, assigned, and transitioned through multiple status changes. The function returned every past version of the task record in chronological order, each entry showing the exact status and assignee at that

point in the task's lifecycle together with the person who made the change and the commit time — from its initial creation with no assignee and status `todo`, through assignment, to its transition to `in-progress`. Because this history is retrieved directly from Fabric's blockchain history rather than a mutable log, no operation exposed by the chaincode or the API allows any of these past entries to be altered or removed.

This behavior is also directly visible in the frontend: the audit trail view (Section 4.7) renders each past record of a task or project as a distinct entry, annotated with what changed, who changed it and when, giving a non-technical user direct visibility into the guarantee the system provides.

The verification procedure of Section 4.6 was exercised on tasks and projects of the demonstration data. For an untouched history the check reports that all records are in committed blocks and shows the head digest; downloading the report and running the stand-alone verifier on it confirms the same digest. In the automated tests, editing any record of a downloaded report, or removing or reordering records, makes the verifier name the first record whose digest no longer matches, and a transaction that the peer does not know is flagged as missing.

5.5 IPFS File Attachment

The IPFS integration described in Section 4.4 was verified by uploading a file through the frontend's file attachment control while viewing a task. The file was uploaded to the local IPFS node via its HTTP API, which returned a content identifier; this CID and the original file name were then submitted to the chaincode's `attachFile` function and became part of the task's on-chain state, visible immediately in the task's attachment list and, in the immutable history, as a new entry in the task's transaction log. Retrieving the file via the IPFS gateway using the recorded CID returned the original file unchanged, confirming that the content-addressing guarantee described in Section 2.6 holds in practice: the CID stored on the ledger correctly and verifiably identifies the exact file that was attached.

5.6 End-to-End Workflow via the Backend API and Frontend

The full request path illustrated in Section 4.1 — frontend to backend API to Fabric Gateway to chaincode, and the corresponding IPFS path for file attachments — was exercised as a whole rather than component by component. Using the frontend interface, an account was created and signed in, a project was created, a second person was added with a role,

a task was added to the project and assigned, the task was progressed through its status workflow, commented on and given a file, and the resulting audit trail was verified, all without direct interaction with the Fabric CLI or the chaincode. Each of these operations completed successfully and the resulting state matched what had been separately verified at the chaincode layer in Section 5.2, confirming that the backend API and frontend correctly and transparently expose the underlying chaincode functionality without introducing discrepancies.

5.7 Automated Tests

The backend has an automated test suite of more than 170 tests, run with Node.js's built-in test runner. The tests run the *real* chaincode source on an in-memory implementation of Fabric's stub interface, behind the real Express application, so no Fabric network is needed. The in-memory ledger reproduces the properties that matter for the tests: a transaction's writes are applied only if the chaincode succeeds (a rejected transaction leaves no partial state), queries never commit anything, each key has a history with transaction identifiers and timestamps, and range scans are ordered by key. The suite covers sign-up and sign-in, sessions, cross-site request protection, rate limiting, password rules (including a check that the browser and the server apply the same rules), Google sign-in, e-mail confirmation, ownership taken from the session, the permission matrix of Table 4.1, the digest chain and audit endpoints, and the project and task rules. The frontend decision logic of every screen (filters, permissions, navigation, refresh behavior, formatting) is covered by unit tests with the same runner, without a browser.

AG: Run `npm test` in both folders just before submitting and give the exact numbers here (backend and frontend test counts).

The in-memory ledger also illustrates the limits of such tests. Two defects were found only when the system was run against the real network: the real Fabric library returns the entries of a history from the newest to the oldest and names their fields differently from the in-memory ledger, which made record numbers, change descriptions and the ledger check wrong until the history functions were corrected (Section 4.2.6). Tests with a simulated ledger are valuable for the business rules, but they do not replace running against the network.

Operation	p50 (ms)	p95 (ms)	Failures
Write, one at a time (`createTask`)	*tbd*	*tbd*	*tbd*
Read one task	*tbd*	*tbd*	*tbd*
Read all tasks of a project (range scan)	*tbd*	*tbd*	*tbd*

Table 5.2: Latency of single operations through the whole stack on a one-machine test network (to be filled in from `benchmark-results.csv`).

5.8 Performance Evaluation

5.8.0.0.1 Method. A benchmark script sends real requests through the backend to the Fabric network, so that every write is a complete transaction: proposal, endorsement, ordering, validation and commit. It creates its own temporary account and project and does not touch other data. It measures four things: the latency of one write at a time (`createTask`); the write throughput at 1, 2, 4, 8 and 16 concurrent requests on different keys; the behavior under contention, with several writers editing the *same* task at once; and the latency of reads of one task and of all tasks of a project. Latencies are reported as the 50th and 95th percentile, together with the number of failures, and each configuration should be repeated and the median run reported.

5.8.0.0.2 Environment.

AG: Fill in: CPU model and cores, RAM, Windows with WSL2 or native Linux, Docker and Fabric versions, number of peers and orderers, and the ordering service's batch timeout (the test network's default is 2 seconds).

5.8.0.0.3 Results. Table 5.2 reports the measurements.

AG: Add a second table of throughput (transactions per second) for 1, 2, 4, 8 and 16 concurrent writers, and the contention result: how many of 8 simultaneous edits of one task succeeded.

5.8.0.0.4 Expected behavior and discussion. The following shape is expected and should be compared with the measured numbers rather than quoted. A write is dominated by the ordering service's batch timeout, so a single write takes about as long as that timeout plus endorsement and commit time; throughput grows with concurrency because several transactions share a block, then flattens. Reads are answered by a peer from its local state and are much faster than writes. When several writers edit the same key simultaneously,

Fabric's optimistic concurrency control commits roughly one write per block and rejects the others with a read conflict; the backend reports this as HTTP 409. This is correct behavior and a design constraint: the data model avoids hot keys by giving every task, project and account its own key.

This agrees with earlier benchmarks of the platform [TNV18], which identify the ordering parameters and key conflicts as the main factors. A conventional database would show lower latency and higher throughput for the same API; this is the price of a tamper-evident history with no single administrator who can rewrite it. The thesis does not include a baseline against a conventional database, which would be a natural extension. The measurement covers the whole stack on a single machine, with the load generator sharing the CPU with the network, and therefore says nothing about a geographically distributed deployment.

5.9 Threat Model

5.9.0.0.1 Trust assumptions. The Fabric network is assumed to be operated honestly enough that no single party controls both endorsement and ordering; the backend is a trusted gateway, since it holds the organization's Fabric identity, verifies the login session and tells the chaincode who is acting; browsers are untrusted.

5.9.0.0.2 What the design protects against. An administrator or database operator who quietly changes a status, a due date or a comment cannot do so without producing a new transaction, and the earlier versions remain in the history; if somebody rewrites the history in a *copy* of the data, the digest chain and the per-transaction check make the alteration detectable, including offline from a downloaded report. Two people cannot claim the same username, phone number or Google account, because uniqueness is enforced by index keys written in the same transaction as the account. A user cannot perform an action that their role forbids even if the backend contains a bug, because the chaincode checks the role of the acting person on every state-changing call.

5.9.0.0.3 What it does not protect against. Table 5.3 lists the residual risks, which are stated openly because they define the limits of the claims made in this thesis.

5.10 Discussion

The results above show that the core requirements identified in Section 2.1 are met by the implemented prototype: projects and tasks can be created and managed through a familiar workflow, membership, roles and assignment are enforced by the chaincode itself rather than by trusted client-side logic, and every change is permanently recorded together with who made it and when, in a form that anyone can re-verify. The IPFS integration additionally demonstrates that file attachments can be incorporated into this guarantee without the scalability cost of storing binary data on-chain.

Three lessons are worth noting. First, the determinism issue discussed in Section 4.2.5 is a general lesson for chaincode development: functionally correct logic can still fail in a multi-peer setting for reasons unrelated to business logic, and this class of bug is easy to introduce and comparatively difficult to diagnose from the resulting error message alone. Second, a testing setup that simulates the ledger differs from the real library in ways that hide defects, as the history ordering shows (Section 5.7). Third, the most important limitation is architectural and not a defect: because the gateway identifies an organization and not a person, the guarantee "only an owner may archive a project" holds relative to the trustworthiness of the backend that reports who is acting (Section 5.9). The evaluation was conducted on a local two-organization network rather than a geographically distributed or production-scale deployment; the implications are discussed as limitations and future work in Chapter 6.

Threat	Why it remains, and possible mitigation
Compromised or malicious backend	Fabric sees only the organization's identity, so the chaincode must trust the acting person reported by the backend; a hacked backend could claim to be the owner. Mitigation: one Fabric certificate per person, so that the chaincode reads the caller from the transaction's client identity.
A malicious organization controlling all endorsers	The test network has two organizations but, in this deployment, a single trust domain. Mitigation: an endorsement policy requiring both organizations for critical transactions.
Stolen password or session	Standard web risk, reduced by scrypt hashing, HttpOnly cookies with hashed tokens, rate limiting, cross-site request checks, and sign-out of other devices on password change.
Unauthorized reading	Reads are not restricted by role: any signed-in user who knows a project identifier can read it. Mitigation: role-checked reads or Fabric private data collections.
Disappearing attachments	Content addressing proves what a file contained but nothing obliges anybody to keep it. Mitigation: pinning and verifying the CID on download.
Permanence of personal data	Data written to the ledger cannot be erased, which conflicts with the right to erasure of the GDPR [Eur16]. Mitigation: keep personal data off-chain or in private data collections and store only hashes on-chain.
Denial of service	A flood of transactions slows ordering. Mitigation: rate limiting at the backend and limits in the ordering service.

Table 5.3: Residual threats and possible mitigations.

Chapter 6

Conclusion

This thesis set out to address a specific limitation of the software project management platforms in wide use today: the absence of an independently verifiable, tamper-evident record of a project's history, stemming from their reliance on a single centralized provider to store and control that history (Section 1.1). Building on the architectural pattern validated by Adel et al. [Ade+23] for decentralized data management in construction (Section 3.1), this thesis adapted that pattern to the software project management domain and implemented it as a working prototype, ChainBoard.

The resulting system models accounts, projects and tasks as chaincode-governed state on a permissioned Hyperledger Fabric network, exposing operations for project creation, role-based membership, task assignment, status transitions, comments and archiving, all recorded in Fabric's immutable transaction history rather than a provider-controlled log (Section 4.2). Who may do what is decided by the chaincode itself, from the roles stored in the project, so the permission rules do not depend on the backend or the user interface (Section 4.2.4). File attachments are stored on IPFS and anchored to the ledger through their content identifiers, avoiding the cost of storing binary data on-chain while preserving a verifiable link between the recorded reference and the exact file content (Section 4.4). A backend API and browser-based frontend expose this functionality through a task-board workflow familiar to users of tools such as Trello or Jira (Sections 4.5 and 4.7), while the system's central guarantee, the audit trail, is made visible and checkable: every change shows who made it and when, and a verification step re-checks each record against the ledger and produces a downloadable report with a digest chain that can be re-verified offline (Section 4.6).

The evaluation in Chapter 5 confirmed that this design works as intended: project and task operations behave correctly and enforce the role rules at the chaincode level, the audit trail correctly preserves every past state of a task or project and can be verified, and file

attachments are correctly anchored to and retrievable from IPFS via the CID recorded on-chain. The evaluation also reports the cost of these guarantees and states what they do not cover. Taken together, these results show that the Fabric-plus-IPFS pattern, previously validated only in a construction-industry context, transfers successfully to software project management without requiring a fundamentally different architecture — only a different data model and user-facing workflow.

6.1 Answers to the Research Questions

RQ1. Yes. The Fabric-plus-IPFS pattern transfers to software project management with a different data model (users, projects, tasks, comments and attachments, with keys designed for a store that has no queries) and a familiar board workflow; all requirements R1–R9 of Table 5.1 are met by the prototype.

RQ2. Yes, with one qualification. The chaincode checks the role of the acting person on every state-changing call, so the rules hold even if the backend or the interface is bypassed, and refused calls leave no trace on the ledger (Section 5.3). The qualification is that the chaincode learns *who* is acting from the backend, because Fabric authenticates the organization and not the person; closing this gap requires one Fabric identity per person.

RQ3. By combining a check of every transaction against the peer with a digest chain over the history, and by letting the user download a report that a stand-alone script re-verifies offline, a person can check a history without trusting the application that displays it (Sections 4.6 and 5.4). The check shows that a history was not altered; it does not show that the person recorded as acting was really the one who acted.

RQ4. Writes cost as much as the ordering service's batch timeout plus endorsement and commit, reads are served by a peer and are much faster, and concurrent edits of the same key conflict (Section 5.8). The limits are stated in Table 5.3 and in the section that follows: identity, confidentiality of reads, permanence of personal data, availability of attachments, and the scale of the evaluation.

6.2 Limitations

The prototype was evaluated on a local, two-organization Fabric network rather than a geographically distributed or production-scale deployment, and the performance figures in Chapter 5 describe one machine under synthetic load, not realistic multi-user use; they also include no comparison with a conventional database. The most important limitation is the identity model: Fabric's gateway authenticates the organization and not each person, so

the backend authenticates people with its own login sessions and tells the chaincode who is acting. The chaincode therefore enforces the *rules* but has to trust the backend about *who* is acting, and a compromised backend could claim any identity (Section 5.9). Reads are not restricted by role, so any signed-in person who knows a project identifier can read it. The data written to the ledger is permanent, which conflicts with the right to erasure of the GDPR [Eur16] for personal data such as names and comments. Attachments are anchored by content identifier, but nothing in the prototype guarantees that the files stay available. The data model is intentionally compact: it does not yet cover sub-tasks, dependencies between tasks, notifications or per-field access control. Finally, the determinism constraint discussed in Section 4.2.5 and the optimistic concurrency of Fabric are reminders that this class of system carries development costs, around correctness patterns specific to multi-peer execution, that centralized platforms do not.

6.3 Future Work

Several directions follow naturally from the limitations above. The most valuable is to give every person their own Fabric identity, with a certificate issued by the Fabric certificate authority, so that the chaincode reads the caller from the transaction's client identity instead of trusting the backend, and to require endorsement by both organizations for critical actions such as archiving a project. Restricting reads by role, or placing sensitive fields in Fabric private data collections and keeping personal data off-chain with only hashes on the ledger, would address confidentiality and erasure. Evaluating the system on a geographically distributed network, with a realistic number of organizations and peers, and comparing it with a conventional database baseline, would allow throughput and latency to be assessed under conditions closer to a production deployment. Extending the data model to support sub-tasks, dependencies and notifications would close the functional gap with mature centralized platforms while preserving the same on-chain audit guarantees, and pinning attachments with verification of their content identifier on download would make the file storage as dependable as the ledger. Finally, since the core architectural pattern evaluated in this thesis is not specific to software projects, applying the same chaincode-plus-IPFS approach to other collaborative domains with similar auditability requirements would help establish how broadly this pattern generalizes beyond the two domains — construction and software project management — in which it has now been demonstrated.

Appendix A

Reproducing the Prototype and its Evaluation

This appendix lists the steps needed to run the prototype and repeat the measurements of Chapter 5. The source code is in the repository named in the project README; the commands assume Linux or WSL2 with Docker, Node.js 20.9 or newer, and the Hyperledger Fabric binaries and `test-network` of the `fabric-samples` repository.

A.1 Starting the Network and the Application

```
cd ~/fabric-samples/test-network
./network.sh down
./network.sh up createChannel -c mychannel
./network.sh deployCC -ccn pmcc -ccp ../dpm-project/pm-chaincode \
    -ccl javascript -ccv 1.0 -ccs 1

cd ~/fabric-samples/dpm-project/pm-backend  && npm install && npm run dev
cd ~/fabric-samples/dpm-project/pm-frontend && npm install && npm run dev
```

The chaincode functions changed signature when permissions were added, so after any chaincode change it is simplest to recreate the network with `./network.sh down`. Demonstration data (five accounts, four projects and their tasks) can be loaded with `npm run seed` in the backend folder.

A.2 Tests and Benchmark

Method	Path	Chaincode function
POST	`/api/projects`	`createProject`
POST	`/api/projects/{id}/members`	`addProjectMember`
DELETE	`/api/projects/{id}`	`archiveProject`
GET	`/api/projects/{id}/tasks`	`getProjectTasks`
GET	`/api/projects/{id}/audit`	`getProjectHistory` + ledger check
POST	`/api/tasks`	`createTask`
PUT	`/api/tasks/{id}/assign`	`assignTask`
PUT	`/api/tasks/{id}/status`	`updateTaskStatus`
PUT	`/api/tasks/{id}/meta`	`updateTaskMeta`
POST	`/api/tasks/{id}/comments`	`addComment`
POST	`/api/tasks/{id}/files`	`attachFile`
DELETE	`/api/tasks/{id}`	`archiveTask`
GET	`/api/tasks/{id}/audit`	`getTaskHistory` + ledger check

Table A.1: Main REST endpoints and the chaincode functions they invoke.

```
cd pm-backend  && npm test             # backend suite, no network needed
cd pm-frontend && npm test             # frontend decision logic
cd pm-backend  && npm run benchmark    # needs the running network and
   backend
npm run verify-audit -- audit-task-<id>.json   # offline check of a
   report
```

The benchmark writes its measurements to `benchmark-results.json` and `benchmark-results.csv`, from which Table 5.2 is filled in.

A.3 Summary of the REST API

Table A.1 lists the main endpoints. All endpoints other than sign-in and sign-up require a valid session cookie, and state-changing requests are checked against the `Origin` and `Host` headers.

Bibliography

[Ade+23] Ahmed Adel et al. "Decentralized Construction Project Data Management Using Blockchain and IPFS". In: *Ain Shams Engineering Journal* (2023).

[And10] David J. Anderson. *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010.

[And+18] Elli Androulaki, Artem Barger, Vita Bortnikov, Christian Cachin, Konstantinos Christidis, Angelo De Caro, David Enyeart, Christopher Ferris, Gennady Laventman, Yacov Manevich, Srinivasan Muralidharan, Chet Murthy, Binh Nguyen, Manish Sethi, Gari Singh, Keith Smith, Alessandro Sorniotti, Chrysoula Stathakopoulou, Marko Vukolić, Sharon Weed Cocco, and Jason Yellick. "Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains". In: *Proceedings of the Thirteenth EuroSys Conference (EuroSys '18)*. ACM, 2018. DOI: 10.1145/3190508.3190538.

[Asa24] Asana Inc. *Asana*. https://asana.com. 2024.

[Atl24a] Atlassian. *Jira Software*. https://www.atlassian.com/software/jira. 2024.

[Atl24b] Atlassian. *Trello*. https://trello.com. 2024.

[Ben14] Juan Benet. *IPFS - Content Addressed, Versioned, P2P File System*. https://arxiv.org/abs/1407.3561. 2014.

[But14] Vitalik Buterin. *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform*. https://ethereum.org/whitepaper. 2014.

[CW09] Scott A. Crosby and Dan S. Wallach. "Efficient Data Structures for Tamper-Evident Logging". In: *Proceedings of the 18th USENIX Security Symposium*. 2009.

[Eur16] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 (General Data Protection Regulation)*. Official Journal of the European Union, L 119. 2016.

[Git24a] GitHub Inc. *GitHub Projects*. https://github.com. 2024.

[Git24b] GitLab Inc. *GitLab*. https://gitlab.com. 2024.

[Hyp24] Hyperledger. *Hyperledger Fabric Documentation*. https://hyperledger-fabric.readthedocs.io. 2024.

[LLK13] Ben Laurie, Adam Langley, and Emilia Kasper. *Certificate Transparency*. RFC 6962, Internet Engineering Task Force. 2013.

[Nak08] Satoshi Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. https://bitcoin.org/bitcoin.pdf. 2008.

[Nat15] National Institute of Standards and Technology. *Secure Hash Standard (SHS)*. Federal Information Processing Standards Publication 180-4. NIST, 2015.

[Per09] Colin Percival. "Stronger Key Derivation via Sequential Memory-Hard Functions". In: *BSDCan 2009*. 2009.

[PJ16] Colin Percival and Simon Josefsson. *The scrypt Password-Based Key Derivation Function*. RFC 7914, Internet Engineering Task Force. 2016.

[Pro21] Project Management Institute. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 2021.

[SK99] Bruce Schneier and John Kelsey. "Secure Audit Logs to Support Computer Forensics". In: *ACM Transactions on Information and System Security* 2.2 (1999), pp. 159–176.

[Sza97] Nick Szabo. *Formalizing and Securing Relationships on Public Networks*. https://firstmonday.org/article/view/548/469. 1997.

[TNV18] Parth Thakkar, Senthil Nathan, and Balaji Viswanathan. "Performance Benchmarking and Optimizing Hyperledger Fabric Blockchain Platform". In: *2018 IEEE 26th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*. IEEE, 2018.

[WG18] Karl Wüst and Arthur Gervais. "Do You Need a Blockchain?" In: *2018 Crypto Valley Conference on Blockchain Technology (CVCBT)*. IEEE, 2018, pp. 45–54.